

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Московский авиационный институт
(национальный исследовательский университет)

Отчёт по лабораторной работе №2

«Управление процессами и межпроцессное взаимодействие»

Выполнила: студентка группы М8О-203БВ-25
Петрова Эвелина
Вариант: 17

Москва, 2026

Содержание

1 Цель работы

Приобретение практических навыков в управлении процессами в операционной системе и обеспечении обмена данными между процессами посредством каналов (pipe).

2 Задание

Составить и отладить программу, осуществляющую работу с процессами и взаимодействие между ними в операционной системе Linux. В результате работы программа должна создать для решения задачи два дочерних процесса. Взаимодействие между процессами осуществляется через неименованные каналы (pipe). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Вариант 17 (Группа вариантов 5): Родительский процесс создает два дочерних процесса. Первой строкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия файла с таким именем на запись для child1. Аналогично для второй строки и процесса child2. Родительский и дочерний процесс должны быть представлены разными программами.

Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1 или в pipe2 в зависимости от правила фильтрации. Процесс child1 и child2 производят работу над строками. Процессы пишут результаты своей работы в стандартный вывод.

Правило фильтрации: строки длины больше 10 символов отправляются в pipe2, иначе в pipe1. Дочерние процессы удаляют все гласные из строк.

Схема межпроцессного взаимодействия приведена на рисунке 1.

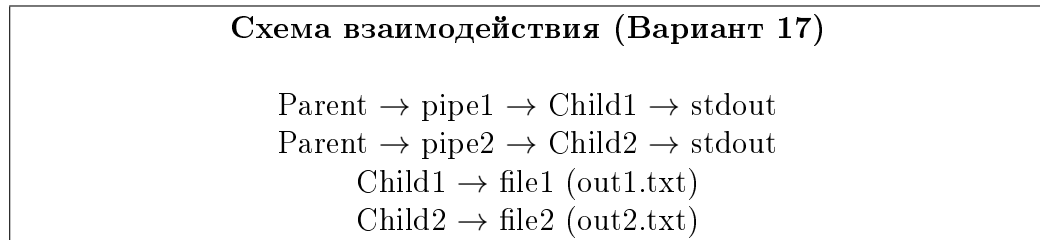


Рис. 1: Схема межпроцессного взаимодействия

3 Теоретические сведения

3.1 Системные вызовы в ОС Linux

Для реализации межпроцессного взаимодействия в POSIX-системах (Linux) используются следующие системные вызовы:

- `pid_t fork()` — создание дочернего процесса. Дочерний процесс является точной копией родителя и наследует все его файловые дескрипторы. Возвращает 0 в дочернем процессе и PID дочернего в родительском.
- `int execve(const char *filename, char *const argv[], char *const envp[])` (и другие вариации `exec`) — замена образа памяти процесса. После успешного вызова текущая программа заменяется новой.

- `pid_t waitpid(pid_t pid, int *status, int options)` — ожидание завершения дочернего процесса. Блокирует вызывающий процесс до тех пор, пока указанный дочерний процесс не завершится.
- `void exit(int status)` — завершение выполнения процесса и возвращение статуса.
- `int pipe(int pipefd[2])` — создание неименованного канала для передачи данных между процессами. `pipefd[0]` — дескриптор для чтения, `pipefd[1]` — для записи.
- `int dup2(int oldfd, int newfd)` — переназначение файлового дескриптора. Используется для перенаправления стандартного ввода/вывода на канал.
- `int open(const char *pathname, int flags, mode_t mode)` — открытие/создание файла.
- `int close(int fd)` — закрыть файловый дескриптор.

3.2 Системные вызовы в ОС Windows

Для полноты описания кроссплатформенного решения приводятся аналогичные вызовы Windows API:

- `BOOL WINAPI CreateProcess(...)` — создание нового процесса.
- `WaitForSingleObject(...)` — ожидание завершения процесса.
- `ExitProcess(...)` — завершение выполнения процесса.
- `CreatePipe(...)` — создание безымянного канала.
- `CreateFile(...)` — создание нового файла.
- `CloseHandle(...)` — закрытие объекта ОС.

3.3 Проблема наследования файловых дескрипторов

Ключевой особенностью системного вызова `fork()` является то, что дочерний процесс наследует *все* открытые файловые дескрипторы родителя. Это означает, что если родитель создал два канала (`pipe1` и `pipe2`), то каждый дочерний процесс получит копии дескрипторов обоих каналов.

Если дочерний процесс не закроет явно дескрипторы «чужого» канала (того, который ему не предназначен), то при закрытии `write-end` канала родительским процессом чтение из `read-end` никогда не вернёт признак конца файла (EOF), поскольку ядро ОС считает канал открытым, пока хотя бы один процесс держит `write-end` открытым. Это приводит к взаимной блокировке (deadlock): дочерний процесс бесконечно ожидает данные, а родительский процесс бесконечно ожидает завершения дочернего через `waitpid()`.

4 Описание программы

4.1 Структура проекта

Проект имеет модульную кроссплатформенную структуру:

- `common/` — общие утилиты: определения платформы (`defines.h`), функции логирования (`errors.h`, `errors.cpp`).
- `os/` — абстракция над API операционной системы. Содержит объявления классов `Pipe` и `ProcessHandle` (`process.h`), общую бизнес-логику (`process.cpp`), а также платформенно-зависимые реализации: `process_posix.cpp` для Linux и `process_win.cpp` для Windows.
- `parent/` — родительский процесс (`main.cpp`). Отвечает за создание каналов, запуск дочерних процессов, маршрутизацию ввода пользователя и ожидание завершения детей.
- `child/` — дочерний процесс (`main.cpp`). Читает строки из стандартного ввода (перенаправленного на канал), удаляет гласные буквы и записывает результат в файл и стандартный вывод.

Система сборки CMake автоматически выбирает нужную реализацию в зависимости от операционной системы:

```
if(WIN32)
    set(OS_SOURCES process.cpp process_win.cpp)
else()
    set(OS_SOURCES process.cpp process_posix.cpp)
endif()
```

4.2 Алгоритм работы родительского процесса

1. Запрос у пользователя имён файлов для `child1` и `child2`.
2. Создание двух неименованных каналов (`pipe1`, `pipe2`) через системный вызов `pipe()`.
3. Запуск двух дочерних процессов через функцию `start_process()`, которая:
 - вызывает `fork()` для создания дочернего процесса;
 - в дочернем процессе перенаправляет `stdin` на read-end нужного канала через `dup2()`;
 - закрывает write-end нужного канала и оба конца «чужого» канала;
 - вызывает `exec1()` для замены образа процесса программой `child`.
4. Чтение строк от пользователя в цикле.
5. Применение правила фильтрации варианта 17: если длина строки > 10 символов — отправка в `pipe2` (`child2`), иначе — в `pipe1` (`child1`).
6. При получении пустой строки — закрытие write-end обоих каналов (для передачи EOF дочерним процессам).
7. Ожидание завершения дочерних процессов через `waitpid()`.

4.3 Алгоритм работы дочернего процесса

1. Получение аргументов командной строки: имя выходного файла и идентификатор процесса.
2. Открытие файла на запись.
3. Чтение строк из стандартного ввода (который перенаправлен на канал родителем).
4. Вызов функции `remove_vowels()` для удаления всех гласных букв (английских и русских).
5. Запись результата в файл и стандартный вывод.
6. Автоматическое завершение при получении EOF (когда родитель закрывает write-end канала).

4.4 Решение проблемы дедлока

Проблема взаимной блокировки решена путём явной передачи ссылки на «другой» канал в функцию `start_process()`. Дочерний процесс принудительно закрывает оба конца «чужого» канала перед вызовом `exec1()`:

```
// В дочернем процессе:  
other_pipe.close_read();  
other_pipe.close_write();
```

Это гарантирует, что после закрытия write-end канала родительским процессом, чтение из read-end корректно вернёт EOF, и дочерний процесс завершит работу.

5 Результаты работы

Программа была скомпилирована и протестирована в среде Linux (WSL Ubuntu). На рисунке 2 показан процесс работы программы.

Как видно из рисунка 2:

- Строки длиной ≤ 10 символов (`hi`, `hello`, `test`) были отправлены в Child1.
- Строки длиной > 10 символов (`hello world`, `programming`) были отправлены в Child2.
- Гласные буквы успешно удалены: `hi` $\rightarrow$ `h`, `hello` $\rightarrow$ `hll`, `programming` $\rightarrow$ `prgrmmng`.
- Оба дочерних процесса завершились с кодом возврата 0.
- Родительский процесс корректно дождался завершения детей и завершил работу.

На рисунке 3 показано содержимое выходных файлов.

```

evelina@WIN-IH2M9MFN8BT:~/Operation-Systems/build$ ./parent/parent
10:32:15 [MSG] Parent: Parent started
Enter filename for Child1: out1.txt
Enter filename for Child2: out2.txt
10:32:24 [MSG] Pipe: Created POSIX pipe
10:32:24 [MSG] Pipe: Created POSIX pipe
10:32:24 [MSG] Process: Child started (PID: 24317)
10:32:24 [MSG] Process: Child started (PID: 24318)

Enter strings (empty line to quit):
10:32:24 [MSG] Child2: Child started
10:32:24 [MSG] Child1: Child started
hi
10:32:29 [MSG] Parent: Line 1 (len=3) -> Child1
[Child 1] h
hello
10:32:35 [MSG] Parent: Line 2 (len=5) -> Child1
[Child 1] hll
hello world
10:32:52 [MSG] Parent: Line 3 (len=12) -> Child2
[Child 2] hll wrld
test
10:33:01 [MSG] Parent: Line 4 (len=6) -> Child1
[Child 1] tst
programming
10:33:08 [MSG] Parent: Line 5 (len=11) -> Child2
[Child 2] prgrmmng

10:33:10 [MSG] Parent: Waiting for children
10:33:10 [MSG] Child2: Child finished
10:33:10 [MSG] Child1: Child finished
10:33:10 [MSG] Parent: Child1 exit: 0
10:33:10 [MSG] Parent: Child2 exit: 0
10:33:10 [MSG] Parent: Parent finished

```

Рис. 2: Консольный вывод программы

```

evelina@WIN-IH2M9MFN8BT:~/Operation-Systems/build$ cat out1.txt
h
hll
tst
evelina@WIN-IH2M9MFN8BT:~/Operation-Systems/build$ cat out2.txt
hll wrld
prgrmmng

```

Рис. 3: Содержимое файлов out1.txt и out2.txt

Выводы

В ходе выполнения лабораторной работы была спроектирована и реализована модульная система межпроцессного взаимодействия на основе неименованных каналов (pipes).

При вызове `fork()` дочерний процесс наследует все файловые дескрипторы родителя. Если не закрыть явно «чужие» концы каналов, EOF не придёт, и программа зависнет. Решение этой проблемы путём принудительного закрытия неиспользуемых дескрипторов в дочернем процессе является ключевым для написания надёжных многопроцессных приложений.

Программа спроектирована так, чтобы работать как в Linux, так и в Windows. Для этого системные вызовы POSIX (`fork`, `pipe`, `dup2`, `waitpid`) инкапсулированы в отдельный файл `process_posix.cpp`, а их Windows-аналоги (`CreateProcess`, `CreatePipe`, `WaitForSingleObject`) — в `process_win.cpp`. Бизнес-логика (удаление гласных, маршрутизация строк) остаётся общей и не зависит от ОС. Это помогает экономить время и ресурсы при портировании приложения на другую платформу — не нужно переписывать весь код, достаточно реализовать платформенно-зависимый модуль. Система сборки CMake автоматически выбирает нужную реализацию в зависимости от целевой операционной системы.